



UNIVERSIDADE DE ÉVORA
CURSO DE ENGENHARIA INFORMÁTICA
2024/2025

Relatório 2.º Trabalho Prático

Sistemas Operativos

Miguel Aleixo 51653
Tiago Morgado 51717
Pedro Freire 52215

1. Introdução

Em C, construímos dois simuladores que funcionam em conjunto: um gestor de memória virtual por paginação, com políticas FIFO e LRU, que traduz endereços lógicos, mantém tabelas de páginas e assinala falhas (SIGSEGV); e um escalonador de processos que executa instruções de LOAD/STORE, SWAP/MEMCPY e JUMPB/JUMPF, gere estados (NEW, READY, RUN, BLOCKED, EXIT) e sinaliza erros como instruções inválidas (SIGILL) e fim de ficheiro (SIGEOF). Ao longo do texto explicamos como estruturámos os dados (tabelas de páginas, filas de substituição e de processos), os algoritmos de alocação e substituição de quadros, e de que forma o gestor de memória e o escalonador interagem para simular o funcionamento de um sistema operativo multitarefa.

2. Estrutura do Programa

Parte 1:

```
|— build
|   |— simulador
|— example
|   |— fifo00(2).txt
|   |— lru00(1).out
|— Makefile
|— output
|   |— fifo
|       |— fifo00.out
|       |— fifo01.out
|       |— fifo02.out
|       |— fifo03.out
|       |— fifo04.out
|       |— fifo05.out
|   |— lru
|       |— lru00.out
|       |— lru01.out
|       |— lru02.out
|       |— lru03.out
|       |— lru04.out
|       |— lru05.out
|— simulador
|— src
|   |— inputs_part1.c
|   |— mem.c
```

Frame

Campos	Descrição
pid	identificador do processo que ocupa o frame
page	Numero da pagina alocada nesse frame
loaded_time	Instante de tempo em que a pagina foi carregada (Usado no FIFO)
last_used_time	Instante de tempo em que a pagina foi acedida pela ultima vez (Usado no LRU)

valid	Indica se o frame esta ocupado(1), livre(0) ou não foi usado usado ainda (-1)
-------	---

Funções Importantes

Funções	Proposito
get_free_frame()	Devolve o indice de um frame livre, se existir.
find_frame(pid,page)	Verifica se uma pagina de um processo ja esta carregada em memoria
choose_victim(algo)	Seleciona o frame a ser substituido com base no algoritmo FIFO ou LRU
simulate	Executa a simulação para um algoritmo dado produzindo o ficheiro de output

Fase de inicialização:

São criadas as pastas de output (output/, output/fifo/, output/lru/) onde os ficheiros com os resultados de cada simulação serão guardados.

O utilizador escolhe uma das 6 simulações pré-definidas (0 a 5), cada uma com:

- Um vetor de **limites de memória por processo** (tamanho máximo em bytes).
- Um vetor com a **sequência de acessos** de cada processo (pares PID, endereço).

O vetor memory[NUM_FRAMES] é inicializado, simulando os 7 frames da memória física, todos livres.

O tempo global current_time é colocado a zero.

É aberto o ficheiro de output correspondente ao algoritmo (FIFO ou LRU).

Fase de simulação:

Para cada par (pid, address) da sequência:

1. Validação de processo e endereço:
 - Se o PID for inválido ou o endereço exceder o limite do processo → é registado um SIGSEGV, e os frames ocupados por esse processo são

desalocados.

2. Cálculo da página:

- A página a aceder é obtida por $\text{page} = \text{address} / \text{FRAME_SIZE}$.

3. Verificação de presença na memória:

- Se a página já estiver carregada (hit), atualiza-se o campo `last_used_time`.
- Se não estiver (page fault), tenta-se:

- Alocar um frame livre.
- Se não houver frames livres, escolhe-se uma vítima com base no algoritmo (FIFO ou LRU), e faz-se a substituição.

4. Atualização e Impressão:

- O estado da memória é impresso no ficheiro de output, indicando que frames estão associados a que processo.
- O tempo global `current_time` é incrementado.

Este ciclo repete-se até serem processadas todas as instruções de acesso.

Fase Final:

O ficheiro de output é fechado.

A simulação é repetida com o segundo algoritmo (se for a execução principal do `main()`).

Ao final, ficam disponíveis dois ficheiros de resultados para comparação: um em `output/fifo/` e outro em `output/lru/`.

Parte 2

```
.
├── build
│   ├── inputs_part2.o
│   ├── main.o
│   ├── mem.o
│   ├── processo.o
│   ├── queue.o
│   └── simulador
├── include
│   ├── mem.h
│   ├── processo.h
│   └── queue.h
├── Makefile
├── output2T00.txt
├── outputs
│   ├── output2T00.out
│   ├── output2T01.out
│   ├── output2T02.out
│   ├── output2T03.out
│   ├── output2T04.out
│   ├── output2T05.out
│   ├── output2T06.out
│   ├── output2T07.out
│   ├── output2T08.out
│   ├── output2T09.out
│   ├── output2T10.out
│   └── output2T11.out
├── simulador.exe
└── src
    ├── inputs_part2.c
    ├── main.c
    ├── mem.c
    ├── processo.c
    └── queue.c
```

Struct Frame

Campos	Descrição
pid	identificador do processo que ocupa o frame
page	Numero da pagina alocada nesse frame
loaded_time	Instante de tempo em que a pagina foi carregada (Usado no FIFO)
last_used_time	Instante de tempo em que a pagina foi acedida pela ultima vez (Usado no LRU)
valid	Indica se o frame esta ocupado(1), livre(0) ou não foi usado usado ainda (-1)

Struct Processo

id	Identificador unico do processo
estado	Estado atual do processo
estado_anterior	Estado anterior do processo
pc	Program counter
instruções	Veror com as intruções do processo
num_instrucoes	Numero de instruções carregadas no processo
tempo_estado	Tempo decorrido no estado atual
block_restante	Numero de ciclos restantes em BLOCKED
quantum	Tempo de execução no Round Robin
tempo_exit	tempo passado no estado EXIT
should_exit	Flag que indica se o processo deve terminar
erro	Codigo de erro associado ao fim do processo
espaco_endereçamento	Tamanho do espaço de endereçamento disponivel para o processo

Main.c (Funções Importantes)

inicializar_memoria()	Inicializa e reinicializa a memoria
aceder_memoria()	Simulação de acessos à memoria pelos processos
imprimir_memoria_estado()	Grava no ficheiro o estado atual da memoria apos cada acesso
libertar_memoria()	Liberta os frames de um processo apos um SIGSEGV
obter_frames_processo()	Gera uma string com os frames usados por um processo para imprimir no output

Processo.c (Funções Importantes)

criar_processo()	Cria dinamicamente um novo processo a partir das instruções fornecidas
atualizar_estado()	Atualiza o estado de um processo consoante o tempo e as regras do ciclo de vida dos processos
executar_instrucao()	Executa a próxima instrução do processo, tratando eventos como bloqueio, saltos, memória, etc
processo_ativo()	Verifica se o processo ainda está ativo (deve ser mostrado no output)

mem.c (Funções Importantes)

inicializar_memoria()	Inicializa e reinicializa a memória física do sistema
aceder_memoria()	Simula o acesso à memória por parte dos processos, incluindo tratamento de page faults
libertar_memoria()	Liberta todos os frames ocupados por um processo, normalmente após um SIGSEGV ou término

obter_frames_processo()	Gera uma string com os frames atualmente usados por um processo para mostrar no output
imprimir_memoria_estado()	Grava no ficheiro o estado atual da memória após cada acesso

Fase de inicialização:

- São criadas as pastas e ficheiros de output necessários, onde os resultados de cada simulação serão guardados.
- O utilizador escolhe um dos 12 inputs pré-definidos (0 a 11), cada um contendo um conjunto de instruções para simular múltiplos processos.
- Os arrays de processos (all_processes) são inicializados, bem como as três filas principais: new_queue, ready_queue e exit_queue, destinadas à gestão dos diferentes estados dos processos.
- O vetor memory[NUM_FRAMES] é inicializado, simulando os 7 frames da memória física, todos livres no início da simulação.
- O tempo global current_time é colocado a zero.
- O primeiro processo é criado e colocado na fila de novos (NEW).
- O ficheiro de output para a simulação é aberto e preparado para receber os resultados.

Fase de simulação:

- A cada ciclo de tempo, os estados dos processos são atualizados de acordo com as suas instruções e eventos (execução, bloqueio, criação, término).
- Processos são movimentados entre as filas conforme a sua evolução (por exemplo, de NEW para READY, de BLOCKED para READY, ou de RUNNING para EXIT).
- Sempre que um processo executa uma instrução de acesso à memória:
- Valida-se o endereço (se exceder o limite do processo, é registado um SIGSEGV e os frames ocupados são libertados).
- Calcula-se a página a aceder ($page = address / FRAME_SIZE$).

- Se a página já estiver carregada em memória (hit), atualiza-se o campo `last_used_time` do frame.
- Se ocorrer um page fault, tenta-se alocar um frame livre; se não houver, escolhe-se um frame vítima pelo algoritmo LRU e faz-se a substituição.
- Eventos especiais como instruções inválidas (SIGILL), finalização de programa (SIGEOF) ou criação dinâmica de processos (EXEC) são tratados de acordo com as regras do enunciado.
- A cada instante temporal, o estado de todos os processos e da memória é impresso no ficheiro de output, permitindo analisar a evolução do sistema.
- O tempo global `current_time` é incrementado a cada ciclo, e o processo repete-se até atingir o tempo limite ou terminarem todas as instruções.

Fase final:

- Após a conclusão da simulação, todos os recursos alocados (processos, filas, e frames de memória) são libertados.
- O ficheiro de output é fechado, garantindo uma terminação limpa do programa e dos resultados produzidos.

3. Conclusão

A implementação deste simulador de sistema operativo em C, agora com suporte à gestão de memória por paginação e substituição de quadros (LRU), permitiu uma análise ainda mais aprofundada sobre o funcionamento interno de um sistema operativo, nomeadamente na interação entre processos e a memória física. O simulador demonstrou um comportamento estável e de acordo com o esperado, refletindo com rigor os princípios teóricos explorados durante o semestre, tanto ao nível do escalonamento de processos como da alocação, utilização e libertação de memória.

Todas as funcionalidades essenciais, desde a execução concorrente de múltiplos processos, o tratamento de faults de memória, a atualização dinâmica dos estados e a correta gestão dos frames, foram corretamente implementadas. Desta forma, o simulador proporcionou uma visão clara e eficaz sobre o ciclo de vida dos processos e sobre as políticas de gestão de memória típicas dos sistemas operativos modernos.